

Mod 5

Due on April 29, 2025

Tuesday/Thursday 11:00-12:15, Warner 209

Mark Floryan - Section 001

Rushil Umaretiya, Lola Quraishi

frj2ka@virginia.edu, uev7gx@virginia.edu

#1 Show that if $P = NP$, then every language $A \in P$, except $A = \emptyset$ and $A = \Sigma^*$ is *NP-Complete*.

In order to show that A is NP-Complete, we need to show that A is in NP and that every language in NP can be reduced to A in polynomial time.

- Show $A \in NP$

This step is trivial, as if $A \in P$, then $A \in NP$.

- Show that every language in NP can be reduced to A.

Given the problem, we know,

$$A \neq \emptyset \implies \exists x \in A \tag{1}$$

$$A \neq \Sigma^* \implies \exists y \notin A \tag{2}$$

Since we know that there are strings that are and are not in A, we can reduce the language to 3-SAT, a known NP-Complete problem.

$$f(\phi) = \begin{cases} w & \text{if } \phi \text{ is satisfiable} \\ y & \text{if } \phi \text{ is not satisfiable} \end{cases}$$

If $P = NP$, then we can solve 3-SAT in polynomial time. We can use the same algorithm to solve A in polynomial time. Thus, we can reduce every language in NP to A in polynomial time. Since 3-SAT is NP-Hard, then we know that A will also be NP-Hard.

Thus, we have shown that A is NP-Complete.

#2 *PSPACE* is the complexity class containing all problems that can be solved using a *Deterministic Turing Machine* that uses at most a polynomial amount of additional space (on the tape). Prove that $P \subseteq PSPACE$. (Hint: Think about a generic problem in P , and the DTM that decides it. Try to use more than a polynomial space with this machine given the allotted time.)

Let A be a language in P . By definition, there exists a DTM M that decides A in polynomial time, let's say this polynomial time is $O(n^k)$ for some constant k .

Since M runs in $O(n^k)$ time, it can execute at most n^k steps before halting. In each step, M can move at most one space to the left or right on the tape. Therefore, by the time M halts, it can only have written on the tape at most n^k symbols.

The space complexity of M is then bounded by n^k which is polynomial in the size of the input.

Since A is decidable by M , and M uses at most n^k space, we can conclude that $A \subseteq PSPACE$.

#3 Recall the *knapsack problem*: Given a list of item values $V = \{v_1, v_2, \dots, v_n\}$, their respective weights $W = \{w_1, w_2, \dots, w_n\}$, a target value $k \in \mathbb{N}$, and a knapsack capacity $C \in \mathbb{N}$, does there exist a subset of items such that the sum of the values of the items is at least k and the sum of the weight of the items is less than or equal to C . In other words, choose items I such that $\sum_{i \in I} (v_i) \geq k$ and $\sum_{i \in I} (w_i) \leq C$. Show that the *knapsack problem* is NP-Complete. You can use any other known NP-Complete problem for your reduction (you might want to look up other NP-Complete problems for this, but you don't have to!).

In order to show that the *knapsack problem*, K is NP-Complete, we need to show that it is in NP and that it is NP-Hard.

- Show that $K \in NP$

Given a solution to K , we can verify in polynomial time that,

$$\sum_{i \in I} (v_i) \geq k \tag{3}$$

$$\sum_{i \in I} (w_i) \leq C \tag{4}$$

Since we are able to verify the problem in polynomial time, we can conclude that K is in NP.

- Show that K is NP-Hard

Let's use the Subset Sum problem for reduction, which is known to be NP-Complete.

The Subset Sum problem is defined as follows: Given a list of integers $S = \{s_1, s_2, \dots, s_n\}$ and a target integer k , does there exist a subset of integers in S such that the sum of the integers in the subset is equal to k ?

Given a Subset Sum instance (S, t) where $S = \{s_1, s_2, \dots, s_n\}$ and t is our target sum, we can construct an instance of the knapsack problem as follows:

- Set the values of the items to be equal to their weights, i.e., $V = W = S$.
- Set the target value $k = t$
- Set the knapsack capacity $C = t$

Now let's show that these problems are equivalent.

Given a solution to the Subset

#4

You are working for *Shoogle* (A data company that definitely does not exist) and you are given the following task from your boss: *Shoogle* is interested in investing in their employees by offering a large amount of professional development. They have collected a series of *workshops* that each cover a subset of *topics*. For example, workshop 1 might cover machine learning and databases, workshop 2 might cover machine learning and data mining, and workshop 3 might cover advanced data structures (note that each workshop can cover multiple topics and there can be overlap across workshops). In addition, *Shoogle* wants to make sure that over the course of the professional development workshops, at least one topic in each sub-area of interest is represented. For example, they might want to make sure there is at least one AI workshop (machine learning or reinforcement learning or neural networks, etc.), at least one systems workshop (databases or cloud computing), etc.

So...the problem is this: Given the availability of the professional development workshop, what topics each covers, and which areas need to be covered, can you devise a schedule over multiple weeks that ensures that at least one topic in every area is covered by at least one workshop?

Let's formalize this a bit: The input contains multiple items:

- T : A list of individual topics that can be covered (e.g., machine learning).
- $W = \{w_1, w_2, \dots, w_m \mid w_i \subseteq T\}$: A list of workshops, each of which is a subset of the topics T that will be covered in that workshop. There are m workshops total.
- $S = \{s_1, s_2, \dots, s_n \mid s_i \subseteq W\}$: The schedule availability for each week. Each s_i is the subset of workshops that can be scheduled in week i . There are n weeks. You can only schedule ONE workshop per week.
- $A = \{a_1, a_2, \dots, a_p \mid a_i \subseteq T\}$: The subject areas that need to be covered. Each area is a list of Topics in that area.

You want to output Yes if there is some schedule of workshops over the n weeks such that every area has at least one topic in T that is covered in at least one workshop.

This problem is *NP-Complete*. Prove it! For your reduction, use *3-SAT*.